ODOO DEVELOPER PROGRAM

Professional Development, Optimization & Deployment

Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

Outline

Introduction

What Will We Learn?

- ORM performance and SQL optimization
- Logging for debugging and operations
- Git & GitHub collaboration workflows
- Docker, Nginx, PostgreSQL backup/restore
- Deployment, upgrades, and migrations
- Code quality and CI/CD concepts

Why This Chapter Matters

- Writing features is only half of professional Odoo work.
- Production systems need speed, safety, backups, and repeatable releases.
- Good engineering habits reduce downtime and technical debt.

Chapter Map

1. Make code fast and observable
2. Manage source with Git/GitHub
3. Package and reverse-proxy with Docker/Nginx
4. Protect data and deploy safely
5. Keep quality high with reviews, tests, and CI/CD

ORM Performance

What Is ORM Performance?

- ORM performance means getting correct results with fewer/faster database operations.
- Common problems:
  - N+1 queries
  - browsing huge recordsets unnecessarily
  - searching without indexes
  - writing in Python loops instead of batch ORM calls

N+1 Query Problem

```python
# Bad: one search + one query per student
students = self.env['student.student'].search([])
for student in students:
    print(student.classroom_id.name)
```

```python
# Better: prefetch / batch read relational data
students = self.env['student.student'].search([])
students.mapped('classroom_id.name')
```

- Prefer batch operations over per-record chatter with the database.

SEARCH VS SEARCH_COUNT VS SEARCH_READ

```python
# Need records? use search
recs = self.env['student.student'].search([('active', '=', True)], limit=80)

# Need only a number? use search_count
total = self.env['student.student'].search_count([('active', '=', True)])

# Need dict rows for API/export? use search_read
rows = self.env['student.student'].search_read(
    [('active', '=', True)], ['name', 'email'], limit=80
)
```

Batch Create / Write / Unlink

```python
# Bad
for vals in vals_list:
    self.env['student.student'].create(vals)

# Good
self.env['student.student'].create(vals_list)

# Bad
for rec in records:
    rec.write({'active': False})

# Good
records.write({'active': False})
```

Indexes and Domains

```
code = fields.Char(index=True)
company_id = fields.Many2one('res.company', index=True)
```

- Index fields used often in search/domain/group by.
- Keep domains selective (`company_id`, `active`, dates, codes).
- Avoid `ilike '%term%'` on huge tables when exact match is possible.

Prefetch and mapped/filtered

```python
students = self.env['student.student'].browse(ids)
# relational prefetch helps when accessing same field on many records
classrooms = students.mapped('classroom_id')
adults = students.filtered(lambda s: s.age >= 18)
```

- Use `filtered`/`mapped` on already-loaded recordsets.
- Use `search` when filtering should happen in SQL.

ORM Performance Checklist

- No N+1 loops on related fields
- Batch create/write/unlink
- Use `search_count` for counts
- Limit fields with `search_read`/`read`
- Index real search fields
- Profile before “optimizing” randomly

SQL Optimization

When to Think in SQL

- Most business code should stay in the ORM.
- Use SQL carefully for:
  - heavy reports
  - complex aggregates
  - performance-critical batch jobs

Rule

ORM first; SQL when measured need exists.

EXPLAIN Your Slow Queries

```
psql school_db
EXPLAIN ANALYZE
SELECT classroom_id, COUNT(*)
FROM student_student
WHERE active = TRUE
GROUP BY classroom_id;
```

- Look for sequential scans on large tables.
- Add/adjust indexes based on real query plans.

Useful Index Patterns

```
CREATE INDEX student_student_company_active_idx
ON student_student (company_id, active);

CREATE INDEX student_student_name_trgm_idx
ON student_student USING gin (name gin_trgm_ops);
```

- Composite indexes help common multi-field domains.
- Trigram indexes can help `ilike` search (when extension is available).

Safe Raw SQL in Odoo

```
self.env.cr.execute("""
  SELECT classroom_id, COUNT(*)
  FROM student_student
  WHERE active = TRUE AND company_id = %s
  GROUP BY classroom_id
""", (self.env.company.id,))
rows = self.env.cr.fetchall()
```

- Always pass parameters with `%s` placeholders.
- Never concatenate user input into SQL strings.

Aggregation Alternatives

```
# ORM read_group for many reporting needs
data = self.env['student.student'].read_group(
    [('active', '=', True)],
    ['classroom_id'],
    ['classroom_id'],
)
```

- Try `read_group` before custom SQL for grouped metrics.

SQL Optimization Tips

- Measure with `EXPLAIN ANALYZE`
- Index for real filters/joins
- Select only needed columns
- Avoid functions on indexed columns in WHERE when possible
- Keep vacuum/analyze healthy on production PostgreSQL

Logging

Why Logging Matters

- Logs explain what the system did and where it failed.
- Useful for:
  - debugging local issues
  - monitoring cron jobs
  - tracing production incidents

Logging in Odoo Python

```python
import logging
_logger = logging.getLogger(__name__)

class Student(models.Model):
  _name = 'student.student'

  def action_archive(self):
    _logger.info("Archiving students: %s", self.ids)
    try:
      self.write({'active': False})
    except Exception:
      _logger.exception("Failed archiving students %s", self.ids)
      raise
```

Log Levels

- `debug`: detailed developer info
- `info`: normal significant events
- `warning`: unexpected but recoverable
- `error`: failure that needs attention
- `exception`: error + stack trace

```
_logger.debug("Domain used: %s", domain)
_logger.warning("Missing email on student %s", student.id)
```

Running Odoo with More Logs

```
./odoo-bin -c odoo.conf --log-level=debug
./odoo-bin -c odoo.conf --log-handler=odoo.addons.school_manager:DEBUG
```

- Increase logs temporarily while investigating.
- Avoid leaving ultra-verbose logging on in production.

What to Log (and Not Log)

- Do log: record IDs, cron start/end, external API status codes
- Do not log: passwords, API secrets, full personal payloads

```
# Bad
_logger.info("Password is %s", password)

# Good
_logger.info("External sync failed for student_id=%s status=%s",
        student.id, response.status_code)
```

Logging Best Practices

- Use `logging.getLogger(__name__)`
- Prefer structured messages with `%s` args
- Log at the right level
- Correlate logs with user actions and cron names
- Rotate and monitor server log files

Git & GitHub

Why Git for Odoo Teams

- Track every module change.
- Collaborate with branches and pull requests.
- Roll back safely when a release fails.
- Review code before it reaches production.

Core Git Workflow

```
git checkout -b feature/student-card
git status
git add .
git commit -m "Add student card report"
git push -u origin feature/student-card
```

- Branch → commit → push → pull request → review → merge.

Useful Daily Commands

```
git pull origin main
git diff
git log --oneline --graph --decorate
git stash
git stash pop
git restore --staged <file>
```

GitHub Pull Request Habits

- Small PRs are easier to review.
- Describe:
  - what changed
  - why it changed
  - how to test
- Request review before merging to main.
- Keep `main` deployable.

What Belongs in Git?

- Yes: custom modules, scripts, deployment configs, docs
- No: credentials, filestore, database dumps, `__pycache__`

```
# .gitignore examples
*.pyc
__pycache__/
.env
*.dump
filestore/
```

Git Best Practices

- Commit often with clear messages
- Never commit secrets
- Protect main branch with reviews
- Use tags/releases for production deployments
- Keep module changes isolated and reviewable

Docker

What is Docker?

- Tool that packages an application and its dependencies into a **container**
- Containers share the host OS kernel — lighter than full VMs
- Same image runs the same way on laptop, server, or CI

Why Odoo teams use Docker

Reproducible environments: same PostgreSQL version, same Odoo version, same Python deps — fewer “works on my machine” bugs.

Core Concepts

Image Read-only template (e.g. `odoo:17.0`)

Container Running instance of an image

Dockerfile Recipe to build a custom image

Volume Persistent storage (database, filestore)

Compose YAML file that starts multiple services together

DOCKER-COMPOSE.YML — DATABASE

```yaml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: odoo
      POSTGRES_PASSWORD: odoo
      POSTGRES_DB: postgres
    volumes:
      - odoo-db:/var/lib/postgresql/data

volumes:
  odoo-db:
  odoo-web:
```

```
web:
  image: odoo:17.0
  depends_on: [db]
  ports:
    - "8069:8069"
  environment:
    HOST: db
    USER: odoo
    PASSWORD: odoo
  volumes:
    - odoo-web:/var/lib/odoo
    - ./addons:/mnt/extra-addons
```

Compose Breakdown

`services`	Containers that make up the stack
`db`	PostgreSQL — Odoo needs a database server
`web`	Odoo application process
`depends_on`	Start order hint (not a health wait by itself)
`ports`	Map host `8069` to container `8069`
`volumes`	Persist DB and filestore; mount custom addons

Everyday Commands

```
# Start in background
docker compose up -d

# Follow Odoo logs
docker compose logs -f web

# Stop (keep volumes)
docker compose stop

# Stop and remove containers (volumes remain)
docker compose down

# Rebuild after Dockerfile change
docker compose build --no-cache
docker compose up -d
```

Custom Image with Your Addons

```
# Dockerfile
FROM odoo:17.0
COPY ./custom_addons /mnt/extra-addons
USER root
RUN pip3 install --break-system-packages \
    python-docx
USER odoo
```

Tip

Keep secrets out of the image. Pass passwords via environment or Docker secrets, never bake them into layers.

Docker Best Practices for Odoo

- One responsibility per container (web vs DB vs reverse proxy)
- Persist PostgreSQL data and Odoo filestore in named volumes
- Pin image tags (`odoo:17.0`, not `latest`)
- Mount addons as volumes in development; bake them into images for production
- Limit container resources (memory/CPU) in production

Nginx

What is Nginx?

- High-performance **web server** and **reverse proxy**
- Sits in front of Odoo (port 8069) and longpolling (8072)
- Handles HTTPS, static files, compression, and load balancing

Why put Nginx in front of Odoo?

Terminate TLS, hide internal ports, serve assets efficiently, and protect the app from direct internet exposure.

Typical Traffic Flow

Browser
↓ HTTPS :443
Nginx
↓ HTTP :8069 / :8072
Odoo workers / longpolling

- Regular HTTP requests → Odoo HTTP port
- Bus / longpolling → gevent longpolling port

Reverse Proxy — Upstreams & Timeouts

```
upstream odoo {
    server 127.0.0.1:8069;
}
upstream odoochat {
    server 127.0.0.1:8072;
}

server {
    listen 80;
    server_name erp.example.com;

    proxy_read_timeout 720s;
    proxy_connect_timeout 720s;
    proxy_send_timeout 720s;
}
```

Reverse Proxy — Locations

```
    location / {
        proxy_pass http://odoo;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For
            $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /longpolling {
        proxy_pass http://odoochat;
    }
```

Directive Breakdown

`upstream`	Named backend pool for Odoo / chat
`proxy_pass`	Forward request to that upstream
`X-Forwarded-*`	Tell Odoo the original host and scheme
`proxy_*_timeout`	Long enough for heavy reports/imports
`/longpolling`	Separate path for realtime bus

HTTPS with Let's Encrypt (Concept)

```
# Install certbot plugin, then:
sudo certbot --nginx -d erp.example.com

# Nginx gains listen 443 ssl and certificate paths
# HTTP can redirect to HTTPS
```

- Prefer TLS everywhere in production
- Redirect port 80 → 443
- Keep certificates renewing automatically

Nginx Tips for Odoo

- Enable `proxy_mode = True` in Odoo config when behind a proxy
- Increase body size for large attachments: `client_max_body_size 64m;`
- Cache or gzip static assets carefully
- Do not expose PostgreSQL or Odoo ports publicly if Nginx is the entry point

PostgreSQL Backup & Restore

Why Backup Matters

- Odoo stores business data in PostgreSQL
- Filestore (attachments) lives on disk — back that up too
- Without tested restores, a backup is only a hope

Rule

A backup you have never restored is not a backup.

What to Back Up

1. **Database** — all Odoo schemas/tables
2. **Filestore** — `$HOME/.local/share/Odoo/filestore/<db>` (or Docker volume path)
3. **Config** — `odoo.conf`, compose files, secrets location notes
4. **Custom addons** — already in Git, but pin the deployed commit

Logical Dump with pg_dump

```
# Custom format (recommended: compressed, parallel restore)
pg_dump -h localhost -U odoo -Fc \
  -f /backups/school_$(date +%F).dump school

# Plain SQL (human-readable, larger)
pg_dump -h localhost -U odoo \
  -f /backups/school.sql school
```

Flags

`-Fc` = custom format; `-f` = output file; last arg = database name.

Restore

```
# Create empty DB first
createdb -h localhost -U odoo school_restore

# Restore custom-format dump
pg_restore -h localhost -U odoo \
  -d school_restore \
  --no-owner --role=odoo \
  /backups/school_2026-07-25.dump

# Plain SQL
psql -h localhost -U odoo -d school_restore \
  -f /backups/school.sql
```

Filestore Backup

```
# Archive filestore next to DB dump
tar -czf /backups/filestore_school_$(date +%F).tgz \
  /var/lib/odoo/.local/share/Odoo/filestore/school

# Restore
tar -xzf filestore_school_2026-07-25.tgz -C /
```

- Database + filestore must stay in sync (same point in time)
- Mismatched pairs cause missing attachments

Odoo Built-in Backup (UI / RPC)

- Database manager: `/web/database/manager`
- Downloads a `.zip` with dump + filestore
- Convenient for small DBs; not ideal as the only production strategy

```
# Automate with cron + pg_dump + tar
0 2 * * * /usr/local/bin/backup_odoo.sh
```

Backup Best Practices

- Automate nightly dumps; keep multiple generations (7 daily / 4 weekly)
- Store copies off-server (S3, another region, cold storage)
- Encrypt backups that leave the machine
- Test restore on a staging host every month
- Document RPO/RTO with the business (how much data loss is acceptable)

Deployment

What is Deployment?

- Moving code and configuration from development to a running production system
- Includes: servers, database, workers, reverse proxy, monitoring, backups

Goal

Ship new features safely, with rollback paths, without surprising users.

Environment Layers

Local	Developer laptop / Docker Compose
Staging	Production-like copy for UAT and migration tests
Production	Live business system

Never

Test migrations or risky modules for the first time on production.

Odoo Service Layout (Bare Metal / VM)

```
# Typical pieces
# 1) PostgreSQL
# 2) Odoo service (systemd)
# 3) Nginx reverse proxy + TLS
# 4) Backups + log rotation

sudo systemctl status odoo
sudo systemctl restart odoo
```

- Run Odoo as a dedicated OS user
- Use `odoo.conf` for db, addons path, workers, proxy mode

Key odoo.conf Settings

```
[options]
admin_passwd = strong-master-password
db_host = 127.0.0.1
db_user = odoo
db_password = ***
addons_path = /opt/odoo/addons,/opt/odoo/custom
proxy_mode = True
workers = 4
max_cron_threads = 1
limit_memory_soft = 2147483648
limit_time_cpu = 600
limit_time_real = 1200
logfile = /var/log/odoo/odoo.log
```

Workers vs Threaded Mode

- `workers = 0` — single process (dev / small)
- `workers > 0` — multiprocess for production concurrency
- Longpolling needs a separate gevent process (often via systemd or compose)
- Size workers from CPU/RAM: rough guide $\approx$ (CPU $\times$ 2) + 1, then measure

Safe Release Checklist

1. Merge to main after review
2. Tag release (`v1.4.0`)
3. Backup DB + filestore
4. Deploy code to staging; run `-u module`
5. Smoke-test critical flows
6. Deploy to production in a maintenance window
7. Watch logs / errors for the first hour

Deployment Best Practices

- Immutable-ish deploys: know exact Git commit on the server
- Separate secrets from code (env files, vault, not Git)
- Prefer rolling or blue/green when downtime is costly
- Keep a written runbook: restart, rollback, restore

Upgrade Scripts & Migration

Two Different Problems

Module upgrade Same Odoo major version; schema/data evolve via `-u`

Version migration Move a database across Odoo majors (e.g. 16 → 17)

Both need backups

Always dump DB + filestore before either path.

Module Upgrade (-u)

```
# Upgrade one module and dependencies as needed
odoo-bin -c /etc/odoo.conf -d school -u school_core --stop-after-init

# Upgrade several
odoo-bin -c /etc/odoo.conf -d school \
  -u school_core,school_fees --stop-after-init
```

- Loads new fields, views, security, and runs migration scripts
- `--stop-after-init` exits after upgrade — good for CI/scripts

Upgrade Scripts (Migration Scripts)

- Live under `migrations/<version>/` in the module
- Run when module version in manifest increases

```python
# school_core/migrations/17.0.1.1.0/pre-migrate.py
def migrate(cr, version):
    # raw SQL before ORM is fully ready
    cr.execute("""
        UPDATE school_student
           SET state = 'draft'
         WHERE state IS NULL
    """)
```

Script Naming & Phases

`pre-*.py`	Before module update (schema may be old)
`post-*.py`	After module update (new schema available)
`end-*.py`	After all modules updated

API

Signature is `migrate(cr, version)` — use cursor carefully; ORM may be limited in `pre-`.

Post-Migrate with Environment

```python
# migrations/17.0.1.2.0/post-migrate.py
from odoo import api, SUPERUSER_ID

def migrate(cr, version):
    env = api.Environment(cr, SUPERUSER_ID, {})
    students = env['school.student'].search([
        ('grade_id', '=', False),
    ])
    default = env.ref('school_core.grade_default',
                      raise_if_not_found=False)
    if default:
        students.write({'grade_id': default.id})
```

Major Version Migration (Concept)

1. Inventory custom modules; port APIs to the new major
2. Upgrade official modules via Odoo upgrade service / OpenUpgrade (community path)
3. Test on a restored copy until scripts are clean
4. Plan cutover: freeze writes, final dump, migrate, switch DNS

- Expect broken XML IDs, renamed fields, removed methods
- Budget serious QA time — this is not a weekend hobby for large DBs

Upgrade / Migration Best Practices

- Bump `version` in `__manifest__.py` when schema/data changes
- Keep migrations idempotent when possible
- Prefer data fixes in scripts over manual SQL on production
- Log progress for long migrations
- Never skip staging for major upgrades

Code Quality

What is Code Quality?

- Code that is correct, readable, testable, and maintainable
- Not only “it works once on my laptop”
- Especially important in Odoo: many modules, shared DB, long-lived projects

Quality Pillars for Odoo Projects

1. Clear module boundaries and naming
2. Security (ACL, record rules, sudo discipline)
3. Performance (prefetch, domains, SQL when needed)
4. Tests (unit / HTTP / tours where valuable)
5. Style consistency (linters, reviews)

Static Checks

```
# Example toolchain (adapt to your repo)
ruff check .
ruff format --check .
pylint-odoo $(find . -name '*.py')

# Odoo module tests
odoo-bin -c odoo.conf -d test_db \
  -i school_core --test-enable --stop-after-init
```

Idea

Catch style and common bugs before review; catch regressions with tests before deploy.

Readable ORM Code

```python
# Prefer clear domains and helpers
def action_enroll(self):
    self.ensure_one()
    if self.state != 'draft':
        raise UserError(_("Only draft students can enroll."))
    self.write({
        'state': 'enrolled',
        'enroll_date': fields.Date.context_today(self),
    })
    return True
```

- Small methods, explicit guards, user-facing errors
- Avoid giant `create`/`write` overrides without comments

Review Checklist (Pull Requests)

- Does it need a migration script?
- New fields: stored? indexed? related?
- Access rights / record rules updated?
- Any N+1 loops or `sudo()` without reason?
- Tests or manual test notes for the change?
- Manifest version bumped when required?

Code Quality Habits

- One PR = one concern
- Delete dead code; avoid commented-out blocks in main
- Document non-obvious business rules near the code
- Prefer extending via inheritance over copy-paste modules

CI/CD Concepts

What is CI/CD?

CI Continuous Integration — every push builds and tests automatically

CD Continuous Delivery/Deployment — tested artifacts ready (or shipped) to environments

Why it matters

Humans forget checklists. Pipelines do not. Faster feedback, fewer broken main branches.

Typical Pipeline Stages

1. **Checkout** — fetch the commit
2. **Lint** — Ruff / pylint-odoo / XML checks
3. **Test** — install modules with `--test-enable`
4. **Build** — Docker image or packaged addons
5. **Deploy** — staging automatically; production on approval

GitHub Actions Sketch

```yaml
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: odoo
          POSTGRES_PASSWORD: odoo
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - name: Run module tests
        run: ./scripts/run_tests.sh
```

What to Automate First

- Lint + format check on every PR
- Install your custom modules on a clean DB
- Run existing Odoo tests for those modules
- Block merge if the pipeline fails

Start small

A short green pipeline that always runs beats a perfect pipeline nobody maintains.

CD Strategies

Manual promote CI builds artifact; human deploys to prod

Staging auto Every main commit deploys to staging

Prod gated Production needs approval / tag / release

- Odoo often needs `-u` during deploy — script it
- Keep DB backups as a pipeline or runbook step before prod upgrade

CI/CD Best Practices

- Secrets in GitHub Secrets / vault — never in YAML committed plaintext
- Cache dependencies to keep pipelines fast
- Same Docker image tag promoted from staging to production
- Notify the team on failure (Slack / email)
- Measure: time-to-feedback and deploy frequency

Summary

What We Covered

1. ORM performance — prefetch, batch, avoid N+1
2. SQL optimization — indexes, EXPLAIN, raw SQL wisely
3. Logging — levels, structured messages, production logs
4. Git & GitHub — branches, reviews, tags
5. Docker — images, compose, volumes
6. Nginx — reverse proxy, TLS, longpolling

What We Covered (cont.)

7. PostgreSQL backup & restore — dump + filestore + test restores
8. Deployment — environments, workers, release checklist
9. Upgrade scripts & migration — `-u`, `migrations/`, majors
10. Code quality — lint, tests, reviews
11. CI/CD — automate check, test, promote

Takeaway Mindset

- Optimize the hot paths you measure
- Treat production as sacred: backup, stage, then ship
- Automate the boring checks so reviews focus on design
- Document runbooks for the 3 a.m. failure

Thank you — questions?